agentic-nlq-eval

Non-Technical Project Documentation

Research harness for comparing AI systems that answer business questions in plain English

Audience: stakeholders, clients, and new team members

Aligned with project version 0.1.0

1. Executive Summary

Imagine you ask a company database a question in everyday language-"Which region made the most profit last quarter?"-and an AI system turns that into a database query, runs it, and explains the answer in words. That capability is often called natural-language querying (NLQ).

agentic-nlq-eval is not a customer-facing product. It is a research test bench: a controlled lab where several AI answering strategies compete on the same questions, the same data, and the same scoring rules. The goal is to learn which approach is more accurate, more trustworthy, and faster as datasets get larger.

Core value proposition: give decision-makers evidence-not opinions-about whether a simple one-shot AI, a retry-on-error AI, or a multi-step agent pipeline is worth the extra complexity and cost for business Q&A over structured data.

In plain terms, the project answers three business questions:

- Does the system get the numbers right? (execution accuracy against known-correct answers)
- Does the written explanation stick to the evidence? (faithfulness of claims)
- How long and how expensive is each approach? (latency and optional spend caps)

Think of it like a standardized driving test for three different car designs: same road, same rules, same scoring sheet-so comparisons are fair.

2. Core Features

2.1 Three answering strategies (plus two variants)

All strategies use the same language model, the same description of the business tables, and the same style of final explanation. That keeps the comparison focused on how the system works, not on which model or prompt library was lucky.

- S1 - Single pass: Writes one database query, runs it, then explains the rows. No second chance. (Analogy: one draft, submit.)
- S2 - Retry: Like S1, but if the query fails checks or execution, the error is fed back. Up to three retries after the first try. (Analogy: teacher marks errors; student revises.)
- S3 - Agentic pipeline: Breaks the job into stages-understand the question, map it to tables, plan the query, write SQL, validate, run, analyze results, write the answer with sources, then verify claims. (Analogy: a checklist of specialist steps.)
- S3-V / S3-A: Ablation variants of S3. S3-V turns off SQL validation; S3-A turns off answer verification-so researchers can see what each check is worth.

2.2 Fair test data and business-friendly tables

The main dataset is TPC-H-synthetic sales and supply-chain data that can be generated at different sizes (from tiny lab samples to tens of millions of line items). The AI never sees cryptic raw table names; it sees business views such as customers, suppliers, sales orders, and order lines. Profit is defined once in one place so every system computes it the same way.

A smaller NYC taxi demonstration set supports a second experiment style: questions over trip data, first on a tiny

fixture for demos and tests, with an optional path to download real public months later.

2.3 Questions with known-correct (gold) answers

Each benchmark question ships with a carefully checked correct answer at a reference data size. Those gold answers are cross-checked on two different query engines so scoring is trustworthy. Questions are original business-style prompts (not copied from the classic TPC-H query list) and are grouped into difficulty tiers.

2.4 Scoring stakeholders can interpret

- Accuracy - Did the system's numbers match the gold answer after fair normalization?
- Faithfulness - When the system writes prose, do its claims match the table it just computed?
- Latency - How long did planning, model calls, and database work take?
- Cost guard (optional) - Experiments can stop before the next AI call once an estimated dollar budget is reached.

2.5 Reproducible experiments and figures

Runs are logged line-by-line to durable files. Order is shuffled with a seed so no system always goes first. A warm-up question is discarded from analysis. Researchers can pause and resume without redoing finished work. A reproduce step turns logs into tidy tables and charts (accuracy comparisons, latency pictures, statistical tests).

2.6 Local demonstration browser (optional)

A small local demo server can show questions and completed run summaries in a browser. It is for inspection and demos; the research core remains a command-line experiment harness, not a hosted SaaS product.

3. User Journey / Workflow

Different people touch the project at different stages. Below is the end-to-end journey from empty laptop to charts leadership can discuss.

Stage A - Set up the lab

- Install the project environment (Python tooling and Java for the big-data engine).
- Copy the example secrets file and place the AI provider key in the local environment only-keys never go into result files or config fingerprints.
- Set the model name and pricing rates in the shared config file so cost estimates are meaningful.

Stage B - Prepare data and questions

- Generate TPC-H Parquet datasets at the needed sizes (small scales for daily work; large scale optional).
- Optionally prepare the NYC demonstration fixture.
- Validate gold SQL so reference answers agree across engines.
- Expand the question bank over time (today's files are a scaffold of example questions per tier).

Stage C - Run a fair competition

Experiment 1 - All TPC-H questions, all systems (including ablation variants), one chosen data size, several repeats.

Experiment 2 - A balanced sample across difficulty tiers, systems S1/S2/S3, and several data sizes (including a very large optional size) to study how performance changes as data grows.

Experiment 3 - NYC taxi questions on the demonstration fixture.

During a run: each system answers each scheduled question; results, timings, and scores are appended to a log. If a budget is set, the harness stops cleanly before starting another model call once spend has hit the cap.

Stage D - Analyze and communicate

- Reproduce figures and summary tables from the logs.
- Compare systems with proper statistics (paired tests on majority-voted repeats, adjusted p-values).
- Discuss trade-offs: S3 may be slower or costlier but more reliable; S1 may be cheapest but brittle; S2 sits in between.
- Optionally browse completed runs in the local demo UI for storytelling.

4. What Success Looks Like

- Leadership can see clear accuracy / faithfulness / latency trade-offs between simple and agentic designs.
- Engineers can reproduce a past run from config fingerprints, prompt versions, and logs.
- Researchers can turn individual checks (SQL validation, answer verification) on or off to measure their contribution.
- Spend stays predictable when a budget guard is enabled.

5. Scope and Current Boundaries

- This is an experiment harness, not a production chatbot or BI product.
- There is no multi-tenant cloud UI; the demo server is local and optional.
- The question bank is intentionally a scaffold until more original questions are authored.
- Live AI experiments require a real model name and API key; automated tests use a scripted fake model so they never spend money.

6. Glossary (plain language)

SQL - The language databases understand. The AI writes it; the engine runs it.

Schema / views - The map of business tables and columns the AI is allowed to use.

Gold answer - The known-correct reference used for scoring.

Faithfulness - Whether the written story matches the computed evidence.

Latency - Wall-clock time from start to answer.

Ablation - Turning one feature off to measure how much it mattered.

JSONL log - A durable diary of runs-one record per line-for later analysis.

Scale factor - How large the synthetic dataset is (bigger = more rows, longer queries).

Document generated from the agentic-nlq-eval codebase for stakeholder onboarding.